

anges tab directly below the merge request title.

1. Hover over line 3 of the left side of the table (which contains the changes in your commit) and click the comment icon.

1. In the comment field, type This is a comment and click Start a review.

> When you start a review, your comments will only be visible to you until you submit the review.

1. You'd normally add more comments to your review before submitting them all at once, but for this lab just click the Finish review button.

1. In the resulting dialog, you will see a Summary comment textbox. This is an optional field that allows you to add a summary to your review. For this review, add the comment Added comment to README.md

1. Below the Summary comment, there are three radio buttons. Since your review only contains general feedback, click Comment.

- Comment: Indicates that the review contains some general feedback. This option will not approve the merge request.

- Approve: Many projects will require a reviewer to approve a merge request before it is merged. This option will submit your feedback, and approve the merge request.

- Request changes: Indicates that the review contains feedback that should be addressed before merging the request.

1. Click the Submit review button.

1. Pretend that someone has seen your comment and added another commit to address it. Click the Resolve thread button to show that the comment has been dealt with.

Task F. Merge the branch and close the merge request

1. Click the Overview tab under the MR title.

1. To mark the merge request as ready to merge, click Mark as ready. This removes Draft: from your MR's title.

> For this lab, you haven't specified any approvers for this project. If you had, an Approve button would appear in the View eligible approvers section.

1. Ensure the Delete source branch checkbox is enabled, and click the Merge button.

1. Navigate back to your project's repository by clicking Code > Repository in the left-hand navigation pane.

1. Check what branch you're on by looking at the dropdown at the top left of the page. Switch

to main if you're not already on it by clicking on the branch name, and clicking the main branch.

1. Since the default view of the repository displays the contents of the README.md file, it's easy to verify that the merge added Edit my README.md file to the contents of README.md.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the lab, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/gitbasichandsonlab4.md

> Estimated time to complete: 30 minutes

Objectives

CI/CD stands for Continuous Integration and Continuous Deployment. In other words, it is a continuous method of software development, where you continuously build, test, deploy, and monitor iterative code changes.

This iterative process helps reduce the chance that you develop new code based on buggy or failed previous versions. GitLab CI/CD can catch bugs early in the development cycle, and help ensure that all the code deployed to production complies with your established code standards.

To use GitLab CI/CD, you start with a .gitlab-ci.yml file at the root of your project which contains the configuration for your CI/CD pipeline. This file follows the YAML format and has its own special syntax. To learn more, see the documentation. It is recommended to bookmark this page for future reference.

Task A. Create a new project and add a CI/CD configuration file

1. Navigate to your My Test Group, click on the New project button, and click the Create blank project tile.

1. In the title, type in CI Test.

1. Set the Visibility Level of the project to Private.

1. Enable the Initialize repository with a README checkbox.

1. Leave Enable Static Application Security Testing (SAST) unchecked.

1. Click the Create project button and wait for GitLab to redirect you to the new project's

main page.

1. Create a new file by clicking (+) > This directory > New file

1. In the Filename dialog box enter `.gitlab-ci.yml`

1. In the Apply a template dropdown, select General > Bash. This populates your file with the contents of a minimal `.gitlab-ci.yml` file.

1. In the editor, delete all lines above the `build1:` line and below the `- echo "For example run a test suite"` line. This will leave you with two sections of code, which define the `build1` and `test1` jobs. Your `.gitlab-ci.yml` file should look like this:

```
yaml
build1:
  stage: build
  script:
    - echo "Do your build here"

test1:
  stage: test
  script:
    - echo "Do a test here"
    - echo "For example run a test suite"
```

> To learn more about jobs, see the documentation.

1. Define build and test stages by adding these 3 lines at the top of the file. The stages keyword must be flush left and the stage names must be indented by 2 spaces.

```
yaml
stages:
  - build
  - test
```

> Use stages to define stages that contain groups of jobs. Use stage in a job to configure the job to run in a specific stage. See the documentation to learn more.

1. Your current `.gitlab-ci.yml` file should look like this:

```
yaml
stages:
  - build
  - test

build1:
  stage: build
  script:
```

- echo "Do your build here"

test1:

stage: test

script:

- echo "Do a test here"
- echo "For example run a test suite"

1. Leave the default values for the Commit message and Target Branch fields, and click the Commit changes button.

Task B. Inspect the CI/CD pipeline

> Pipelines are the top-level component of continuous integration, delivery, and deployment. Pipelines comprise of Jobs, which define what to do, and Stages, which define when to run the jobs. For example, a stage can be defined for all the operations that build a project, and a job of this stage could be to compile the code. To learn more, click [here](#).

1. GitLab started running a CI/CD pipeline as soon as you committed `.gitlab-ci.yml` to your project's repository. To see the project's pipelines, navigate to Build > Pipelines.

1. Only 1 pipeline has run so far, so your table of pipelines has only 1 row. See the details of that pipeline by clicking the status label at the left of the pipeline's row.

> The status label should show either Running or Passed

1. Inspect the pipeline graph. Each column represents a stage. In the Build stage, there's a widget representing the build1 job. In the Test column there's a widget representing the test1 job. Click on the build1 widget to see the job's output in a web terminal. For example, look for the message Do your build here in the output.

1. Return to the pipeline graph by clicking on the back button on your web browser. Click on the test1 widget to see the job's output in a web terminal. For example, look for the message Do a test here in the output.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the lab, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/gitbasichandsonlab5.md

> Estimated time to complete: 30 minutes

Objectives

GitLab Auto DevOps is a collection of pre-configured features and integrations that work together to support your software delivery process. Auto DevOps detects your programming language and uses CI/CD templates to create and run default pipelines to build and test your application. Then, you can configure deployments to deploy your apps to staging and production, and set up Review Apps to preview your changes per branch. See the documentation to learn more.

In other words, Auto DevOps is an alternative to writing and using your own `.gitlab-ci.yml` file.

Task A. Create a new Node JS Express project with Auto DevOps

> We will use a pre-defined template for NodeJS Express to show how Auto DevOps works. Pre-defined templates allow you to start off with a base project instead of starting from scratch. A list of all of our base projects can be found [here](#).

1. Navigate to your My Test Group and select New project,.
1. Instead of making a blank project, click on the Create from template tile.
1. Click the Use template button next to NodeJS Express.
1. In the Project name field, enter Auto DevOps Test Project
1. Make sure the Visibility Level is Private, and then click the Create project button.
1. At the top of your project, there is a banner with the title Auto DevOps. Below this banner, click the Enable in settings button.
 - > If this banner does not appear for you, navigate to Settings > CI/CD, click the Expand button next to Auto DevOps.
1. Click the Default to Auto DevOps pipeline checkbox.
1. For Deployment strategy, click Automatic deployment to staging, manual deployment to production.
 - > For your own projects, you may choose to use a different deployment strategy. To learn more about each strategy, click the blue question mark beside each option.
1. Click the Save changes button.
1. In the left-hand navigation pane, click Code > Branches.
1. In the branches menu, click New branch.
1. In the Branch name field, enter new-feature.
1. Ensure that the Create from branch is set to master.

1. Click the Create branch button.

1. In the left-hand navigation pane, click on Build > Pipelines. You'll see an Auto DevOps pipeline running on the branch you just created.

1. Click on the pipeline's running status icon and note the stages (represented by columns in the pipeline graph) and the jobs that Auto DevOps has created.

> You may see the dast job fail in your pipeline. This job requires additional configurations to scan successfully. See the documentation to learn more about DAST scan configurations.

Task B. Commit a change to trigger a pipeline run

> The most common way to run a pipeline is to commit to a branch in your project's repository. In this section, you will apply a new commit and view the resulting pipeline.

1. Navigate to Code > Repository.

1. Near the top left of the window, switch to the new-feature branch by selecting it in the dropdown that currently says master.

1. In the list of repository files, click the server.js file.

1. Click Edit > Edit single file and modify the get endpoint as follows:

```
js
app.get("/", (req, res) => {
  return res.status(200).send({
    message: "Hello World from GitLab!",
  });
});
```

1. The file should now look like this:

```
js
const express = require("express");
const app = express();

const port = process.env.PORT || 5000;

app.get("/", (req, res) => {
  return res.status(200).send({
    message: "Hello World from GitLab!",
  });
});

app.listen(port, () => {
  console.log("Listening on " + port);
});
```

```
});
```

```
module.exports = app;
```

1. For Commit message, type Update welcome message in server.js

1. Leave Target branch set to new-feature

1. Click on the Commit changes button.

After you commit these changes, a pipeline will run, and the test stage will fail. This is because the test cases no longer match the contents of the index file. To ensure that the tests in our pipeline run successfully, we will also need to update our tests to match the new index file.

1. In the list of repository files, click the tests directory and then the server.test.js file.

1. Click Edit > Edit single file and modify the line `assert.equal(res.body.message, 'Hello World!');` to `assert.equal(res.body.message, 'Hello World from GitLab!');`. After completing the edits, your code will look like this:

```
js
const request = require('supertest');
const assert = require('assert')
const app = require('../server');

describe('GET /', () => {
  it('responds responds to the world', async function() {
    const res = await request(app)
      .get('/')
      .set('Accept', 'application/json');

    assert.equal(res.status, 200);
    assert.equal(res.type, 'application/json');
    assert.equal(res.body.message, 'Hello World from GitLab!');
  });
});

describe('GET /404', () => {
  it('responds with a 404', async function() {
    const res = await request(app)
      .get('/404')
      .set('Accept', 'application/json');

    assert.equal(res.status, 404);
  });
});
```

1. For Commit message, type Update welcome message test

1. Leave Target branch set to new-feature

1. Click on the Commit changes button.

1. Once the change have been committed, click on the Create merge request button.

1. Click the Mark as draft checkbox to set the merge request to draft.

> If you type DRAFT: at the front of the title, the Mark as draft checkbox will check automatically.

1. Assign the merge request to yourself by clicking on the Assign to me option next to Assignees.

1. Leave all other fields at their default values and click Create merge request at the bottom of the page.

1. To mark the merge request ready to merge, click the Mark as ready button. This removes Draft: from your MR's title.

> You now have an active merge request for merging the new-feature branch into the master branch. The page you are on shows the details of that merge request, including the status of the last pipeline that was run on the new-feature branch (you might have to refresh the page to see the pipeline status). GitLab will run a new pipeline every time you commit to the new-feature branch.

1. The Auto DevOps pipeline automatically executes with your merge request. This pipeline contains various stages and jobs. As the pipeline runs, you will see new sections appear inside of your MR. Once the pipeline completes, refresh the page to see the results.

> When your pipeline completes, it may display a warning status. The warning is due to the dast scan not being configured. For this example, it is ok to proceed with this warning, since we will not use dast.

1. Select your merge request pipeline

1. In the pipeline details, you will see three stages associated with the Auto DevOps pipeline:

- The build stage, which creates a build of the application using an existing Dockerfile or Heroku buildpacks. The resulting Docker image is pushed to the Container Registry, and tagged with the commit SHA or tag. Once the build stage is completed, navigate to Deploy > Container Registry to see your application.

- The test stage, which runs various tests on your application code to ensure it is secure and high quality. A few of the jobs are explained below:

- The Dependency scan will scan to detect any new licenses and dependencies added to the

project. Select Full report in the License Compliance section to see the scan details. After viewing the report, return to the pipeline.

- The Code Quality scan will scan to detect if the code quality has changed between the main and the merge request code. If any code quality issues exist, they will be flagged in this section.

- The SAST, Dependency, Secret Detection, and Container Scan jobs will check if any new vulnerabilities have been introduced in the code. Select View all pipeline findings in the Security Scanning section to see the scan details.

- The dast stage, which uses a running version of your application to check if there are any known vulnerabilities that it can find by running API calls on your application. Since we did not configure a live environment for the DAST job to scan, we do not need to worry about this job.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the lab, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/gitbasichandsonlab6.md

> Estimated time to complete: 30 minutes

Objectives

This lab uses SAST, an optional feature in CI/CD pipelines, to identify security vulnerabilities in your code. GitLab's Vulnerability Report then shows any old or new vulnerabilities found with each pipeline run. You can learn more in the documentation.

Task A. Enable SAST in your CI Test project

1. Navigate to your CI Test project.

1. Click on the .gitlab-ci.yml file and click Edit > Edit single file.

1. Copy the following lines at the bottom of your gitlab-ci.yml file:

```
yaml
include:
  - template: Jobs/SAST.gitlab-ci.yml
```

> To learn more about integrating SAST scanning into your .gitlab-ci.yml file, see the documentation.

1. The current .gitlab-ci.yml file should look like this:

```
yaml
stages:
  - build
  - test

build1:
  stage: build
  script:
    - echo "Do your build here"

test1:
  stage: test
  script:
    - echo "Do a test here"
    - echo "For example run a test suite"

include:
  - template: Jobs/SAST.gitlab-ci.yml
```

> The include lets you include external YAML files in your CI/CD configuration. You can split one long .gitlab-ci.yml file into multiple files to increase readability, or reduce duplication of the same configuration in multiple places. You can read more about the include keyword in the documentation.

1. Enter an appropriate Commit message.

1. Set the Target Branch to main.

1. Click the Commit changes button.

Task B. Add run.py and review SAST scanning results

In this task, you'll add a file with known vulnerabilities and see if SAST detects it.

1. Return to the Project overview page by clicking on the name of your project in the breadcrumbs section.

1. At the top of the project landing page, to the right of the branch dropdown, click (+) > This directory > New file.

1. For the File name field, type in run.py.

1. Copy the content below into the file:

```
python
import subprocess

in = input("Enter your server ip: ")
subprocess.run(["ping", in])

print("Attempting to connect to the server")
print("Application authentication was successful")
```

1. Add an appropriate Commit message.

1. Set the Target Branch to main.

1. Click the Commit changes button.

1. In the left-hand navigation pane, click Build> Pipelines.

1. At the top of the row of the table of pipelines, click on the running (if it is still running) or passed (if the pipeline has been completed) status labels.

> The SAST scan may take a few moments, so feel free to grab a cup of coffee while you wait.

1. When the pipeline finishes, in the left navigation pane, click on Secure > Vulnerability report.

1. Click any of the vulnerabilities and read about a potential security problem detected by SAST scanning in run.py.

1. Feel free to edit the code to fix the issue raised (such as removing the subprocess.run command), and commit the changes. Does the vulnerability report still note the issue as present?

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the lab, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/gitlabci/cdhandson.md

GitLab CI/CD Lab Guides

| Lab Name | Lab Link |

|-----|-----|
| Understanding the Basics of Pipelines | Lab Link |
| Defining Stages, Jobs, and Runners | Lab Link |
| Create a basic CI configuration | Lab Link |
| Display Pipeline Info | Lab Link |
| Variable Hierarchy | Lab Link |
| Job Policy Patterns | Lab Link |
| Using Artifacts | Lab Link |
| GitLab Docker Registry | Lab Link |
| Security Scanning | Lab Link |
| Code Quality Scanning | Lab Link |

Quick links

Here are some quick links that may be useful when reviewing this Hands-On Guide.

[GitLab CI/CD Course Description](#)
[GitLab CI/CD Specialist Certification Details](#)

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/gitlabci cdhandsonlab1.md

> Estimate time to complete: 15 - 20 minutes

Objectives

CI/CD is a continuous method of software development, where you continuously build, test, deploy, and monitor iterative code changes. To achieve these tasks, GitLab uses various different components. The objective of this lab is to familiarize yourself with the GitLab Pipeline UI, as well as the jobs and stages that make up a pipeline.

Task A: Review GitLab's CI/CD Pipelines

1. Click [here](#) to navigate to GitLab's pipelines. You will see all of the pipelines that are running for commits to the main GitLab repo.

On this page, you will see a table with four columns. The four columns you see at the top are:

Status: The current status of the overall pipeline. Each job in the pipeline will have their own status as well.

Pipeline: The name of the pipeline being ran. Also contains information on what merge

request and commit the pipeline is running on.

Created by: The individual responsible for starting the pipeline.

Stages: The phases of the pipeline.

1. Hover over any of the icons in the Stages column to see the status of each stage as the pipeline progresses.

Some common statuses you can see are:

Passed: The job has successfully completed.

Running: The job is actively processing.

Created: The job is waiting on other jobs to complete before it can run, but it is ready to run.

Failed: The job has not passed due to an exception. This does not necessarily mean that the entire pipeline will fail.

1. Click on any of the running pipeline statuses in the leftmost column to view that pipeline's graph. The graph is a visual representation of the order of stages in a pipeline, as well as the statuses of the jobs in a stage. By default, all jobs in a stage must succeed in order for the next stage to run.

> There are exceptions to the rule that all jobs in a stage must succeed for the next stage to run. You can read more about the exceptions [here](#).

1. Review the names of the pipeline's stages. The name of a stage can be whatever you want it to be. The order of the stages will be determined by the `.gitlab-ci.yml` file.

1. Click on the name of a job with either a Passed or a Running status to see the output from the jobs.

> Each job runs in an environment independent of all other jobs.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/gitlabci cdhandsonlab2.md

> Estimate time to complete: 15 - 20 minutes

Objectives

In this lab, you'll enable CI/CD for a GitLab project. After creating your first `.gitlab-ci.yml` file, you will explore the CI/CD pipeline to better understand jobs and stages. Finally, you will learn how to install, run, and register a runner with a GitLab instance.

> Parts D through G in this exercise require admin rights to your local machine. If you are unable to install GitLab Runner locally, you may skip parts D through G and use the training environment's shared runners instead.

Task A. Access your Lab Environment

1. Navigate to <https://gitlabdemo.com/invite> in a web browser.

1. In the Invitation Code field, enter the invitation code provided by your instructor or in the LevelUp LMS.

1. Select Provision Training Environment.

1. For self-paced courses, you will need your GitLab.com username. To find your username, navigate to Gitlab.

1. In the left sidebar, select your profile image.

1. In the resulting dropdown, you will see a value starting with @. This is your GitLab username.

1. After inputting your invitation code, the portal prompts you for your GitLab.com username. Enter your GitLab.com user (excluding the leading @ sign) in the field provided.

1. Select Provision Training Environment.

1. Select My Group at the bottom of the page.

1. Sign in with your GitLab.com credentials.

1. You will be redirected to a My Test Group group that provides a sandbox for you to perform training lab steps in.

> This group has a GitLab Ultimate license to see all of the features while your personal username namespace requires a paid subscription or a free trial to access all of the features.

>

> If you receive a 404 error when accessing your group, it is likely due to the username input during the lab provisioning. Double check your GitLab username to make sure it is entered correctly.

1. From your My Test Group training subgroup, click the New project button.

1. Continue to Task B.

Task B: Create a Project

1. From the My Test Group training subgroup, click the New project button.

1. Click the Create blank project tile.

1. In the Project name text box, enter CICD Demo.

> The project slug will automatically populate. You can change this to a shorter string if desired for your own project. Leave it at the default for this lab.

1. In the Project URL field, click the dropdown for the second half of the URL to make sure it's pointing to a group name (starts with gitlab-learn-labs/ if it is SPT, and training-users/ if it is ILT) and not a username. You should create this project inside a group, not directly in your user's namespace.

1. Under Visibility Level, ensure Private is selected.

> Since the parent group above your group is private, all child groups and projects below will be private. You can learn more about project visibility levels in the documentation.

1. Check Initialize repository with a README.

1. Ensure that the Enable Static Application Security Testing (SAST) checkbox remains unchecked.

1. Click the Create project button.

Task C. Add a .gitlab-ci.yml File

> To use GitLab CI/CD, you start with a .gitlab-ci.yml file at the root of your project. The .gitlab-ci.yml file contains the configurations for your CI/CD pipeline. In this section, you will create a simple gitlab-ci.yml file.

1. Create a new file in the main branch by clicking (+) > This directory > New file.

1. In the Filename field, type .gitlab-ci.yml

1. Select the Bash template from the Apply a template dropdown. This will pre-populate the file.

1. To create a minimal .gitlab-ci.yml file:

Delete all lines above build1.

Delete all lines below echo "For example run a test suite" in the test1 section.

1. Add build and test stages by pasting these lines at the top of the file.

yml

stages:

- build
- test

> Keep in mind that YAML files should be indented with two spaces. Your web IDE may try to use a tab with 4 spaces. Simply use the backspace to set 2 spaces if you are not copying and pasting the examples.

1. Set the commit message to Creating a simple .gitlab-ci.yml file, and set the Target Branch to main.

1. Click Commit changes.

1. After committing the changes, you will have a .gitlab-ci.yml that looks like this:

```
yml
stages:
  - build
  - test

build1:
  stage: build
  script:
    - echo "Do your build here"

test1:
  stage: test
  script:
    - echo "Do a test here"
    - echo "For example run a test suite"
```

This file defines two stages: build and test. The build1 job executes during the build stage, running all of the commands in script. The test1 job executes during the test stage, running all of the commands in script.

Task D. View a Pipeline's Status, Stages, Jobs, and GitLab Runner

> When you commit your .gitlab-ci.yml file, a pipeline is created. A pipeline comprises of jobs and stages. In the previous section, you defined two stages: build and test. Each of these stages contained jobs, which were defined in script. In this section, you will view the pipeline created from your .gitlab-ci.yml file.

1. In the left navigation pane, click Build > Pipelines to see an overview of all pipelines. The top row in the overview shows the pipeline that started a few seconds ago, when you committed .gitlab-ci.yml. The status icon at the left of the row should say either running or passed.

1. Click the status icon of the top row to see the details of the most recent pipeline. You'll

see columns representing the pipeline's stages, and widgets representing jobs within each stage.

> The order of execution for stages generally reads left to right. In this example, the build stage is the leftmost column, since it is the first stage to execute.

1. Click each of the two jobs to see the output in a web terminal. Identify the gitlab-runner for each job

> Hint: it's listed near the top of each job's output.

Task E. Prepare to Install GitLab Runner Locally

> Jobs are executed by runners. If your project is hosted on GitLab.com, various SaaS runners are provided to build, test, and deploy your application on different environments. In some cases, you may want to host your own runners. Sections D, E, and F will outline the process of installing and registering a runner on your GitLab instance.

1. Depending on which OS you're on, run the appropriate command(s):

In a Linux terminal:

```
sh
sudo gitlab-runner status
```

In a macOS terminal:

```
sh
gitlab-runner status
```

In a Windows PowerShell window:

```
powershell
cd C:\GitLab-Runner
.\gitlab-runner.exe status
```

1. If the command gives an output like: gitlab-runner: Service is running, then you already have a runner installed on your system. If a runner is already installed on your system, skip to Part F below. If the command throws an error, continue with the next section.

Task F. Install the GitLab Runner Binary on your Computer

> This section outlines the steps required to install a GitLab runner on your computer. Follow only the instructions that match the operating system you're using.

Linux

1. Follow steps 1 and 2 only in this documentation.

1. Verify that the gitlab-runner service has started by running this command:

```
sh
sudo gitlab-runner status
```

If you see Service is running in the output, the gitlab-runner service is working as expected.

macOS

1. Follow steps 1 and 2 only in this documentation.

1. Install gitlab-runner as a service and start the service:

```
sh
cd ~
gitlab-runner install
gitlab-runner start
```

1. Verify that the gitlab-runner service has started by running this command:

```
sh
gitlab-runner status
```

If you see Service is running in the output, the gitlab-runner service is working as expected.

Windows

1. Follow steps 1 and 2 only in this documentation.

1. Open an elevated PowerShell window:

1. Click Start.

1. Type PowerShell

1. Right-click Windows PowerShell.

1. Click Run as administrator.

1. From the elevated PowerShell window, install and start the gitlab-runner service:

```
powershell
cd C:\GitLab-Runner
.\gitlab-runner.exe install
.\gitlab-runner.exe start
```

1. Verify that the gitlab-runner service has started by running this command:

```
powershell  
.\gitlab-runner.exe status
```

If you see Service is running in the output, the gitlab-runner service is working as expected.

Task G. Register a Specific Runner Dedicated to your Project

> At this point, you have installed a runner on your system. To allow GitLab to use this runner for CI/CD jobs, you need to register the runner in the UI.

1. In your CICD Demo project, in the left navigation pane, click Settings > CI/CD.

1. Scroll down to the Runners section. Click the Expand button next to it.

1. Under the Project runners section, click the New project runner button.

1. Under Tags, select Run untagged jobs. Leave the rest of the options blank.

> An untagged runner will run any jobs. To control the jobs that a runner can run, you can define tags for the runner. To learn more about this process, [click here](#)

1. Click the Create runner button.

1. From the next page presented, select your operating system (Linux, MacOS, or Windows).

1. Copy the command under Step 1 to your clipboard.

1. Back in your terminal, paste and run the command you copied in the previous step. Press Enter for the instance URL and Enter for the runner name to use the default values. (You can give this a custom name if desired).

1. When prompted for the executor, enter shell

1. Confirm that your gitlab-runner registered correctly by running the appropriate command(s) for your OS:

In a Linux terminal:

```
sh  
sudo gitlab-runner list
```

In a macOS terminal:

```
sh  
gitlab-runner list
```

In a normal (not elevated) Windows PowerShell window:

```
powershell  
cd C:\GitLab-Runner  
.\gitlab-runner.exe list
```

> If your runner is registered correctly, you should see an output like this:
gitlab-runner run Executor=shell Token=your-gl-token URL=https://gitlab.com

1. If you're on Windows, follow these additional instructions to configure your gitlab-runner to use the right command to start PowerShell:

1. Open C:\GitLab-Runner\config.toml in a text editor.

1. Change this line:

```
toml  
shell = "pwsh"
```

to this:

```
toml  
shell = "powershell"
```

1. Save the file.

(Optional) Task H: Unregistrering a GitLab Runner

If you do not wish to keep the GitLab Runner on your device, you may follow the steps below to unregister the GitLab Runner.

1. Get the URL and the token of the GitLab Runner by running the following command (dependent on your OS):

In a Linux terminal: `sudo gitlab-runner list`

In a macOS terminal: `gitlab-runner list`

In a normal (not elevated) Windows PowerShell window:

```
ps  
cd C:\GitLab-Runner  
.\gitlab-runner.exe list
```

1. Using the URL and token, run the unregister command: `gitlab-runner unregister --url "https://gitlab.example.com/" --token t0k3n`

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/gitlabci cdhandsonlab3.md

> Estimate time to complete: 15 - 20 minutes

Objectives

The .gitlab-ci.yml file allows you to define the stages and jobs for your CI/CD process. In this lab, you will learn how to modify your .gitlab-ci.yml file.

Task A. Define a Basic .gitlab-ci.yml File

1. Open your CICD Demo project from the last lab.

1. On the left Navigation pane click Code > Repository. Click on your .gitlab-ci.yml file to view its contents. Click Edit > Edit single file. Replace all the code in .gitlab-ci.yml with the content of the following snippet:

```
yml
stages:
  - test
  - build

test job:
  stage: test
  script:
    - echo "I am a unit test!"

build job:
  stage: build
  script:
    - echo "I am a build image!"
```

> The pipeline logic will be almost identical to what you had previously, just the job names and echo statements will change slightly.

1. In the Commit message field, type Add CI starter, set the Target Branch to main, and click Commit changes.

1. Refresh the page to make the pipeline status icon appear. Check that the configuration is

valid and that the pipeline is running by hovering over the Pipeline: running icon or the Pipeline: passed icon in the upper right corner of the page, to the left of the commit's SHA.

1. When the pipeline status changes to the Pipeline: passed icon, click it to review the pipeline graph for your CI configuration.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/gitlabci cdhandsonlab4.md

> Estimate time to complete: 15 - 20 minutes

Objectives

When a job is executed in a pipeline, it runs inside of an environment. To ensure that environments are set up to run a job successfully, a pipeline defines various environment variables. In this lab, you will learn how to display the environment variables for your pipeline.

Task A. Viewing your Environment Variables

1. Open your CICD Demo project from previous labs.

1. Click on your .gitlab-ci.yml file to view its contents. To edit the file, click Edit > Edit single file.

1. Paste the contents of the following snippet at the bottom of .gitlab-ci.yml. There should be an empty line between the file's previous content and the pasted snippet content.

```
yaml
environment echoes:
  stage: build
  script:
    - echo "Who am I running as..."
    - whoami
    - echo "Where am I..."
    - pwd
    - ls -al
    - echo "Here's what is available in our environment..."
    - env
```

1. After pasting the contents, your .gitlab-ci.yml file will look like this:

```
yml
stages:
  - test
  - build

test job:
  stage: test
  script:
    - echo "I am a unit test!"

build job:
  stage: build
  script:
    - echo "I am a build image!"

environment echoes:
  stage: build
  script:
    - echo "Who am I running as..."
    - whoami
    - echo "Where am I..."
    - pwd
    - ls -al
    - echo "Here's what is available in our environment..."
    - env
```

1. In the Commit message field, type Display pipeline environment variables, and set the Target Branch to main

1. Click the Commit changes button.

1. In the left-hand navigation pane, click Build > Pipelines. The pipeline that was started when you committed changes is represented by the first row in that table.

1. After the pipeline completes, click the status icon at the left of the first row to see the pipeline's jobs. Click the environment echoes job and review its output to see the environment variables on the runner.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/gitlabci cdhandsonlab5.md

> Estimate time to complete: 15 - 20 minutes

Objectives

To customize your CI/CD process, you can define your own environment variables. In this lab, you will learn how to define inline global variables, inline local variables, and group and project level variables.

Task A: Add Inline Variables

There are two types of inline variables we will explore in this section: global inline variables and job scoped inline variables. These variables are defined only for the .gitlab-ci.yml file they are declared in.

Variables in GitLab CI/CD have a precedence, which means variables at a higher 'level' will override the values of a lower 'level'. This can lead to unintended results, so re-use of variable names should be monitored carefully. For more information, [click here](#).

1. Open your CICD Demo project from previous labs.

1. Click your .gitlab-ci.yml file to view its contents. To edit the file, click Edit > Edit single file.

1. Paste the following snippet at the end of the file, with an empty line between the file's previous content and the snippet's content.

```
yml
environment echoes:
  stage: build
  script:
    - echo "Who am I running as..."
    - whoami
    - echo "Where am I..."
    - pwd
    - ls -al
    - echo "Here's what is available in our environment..."
    - env

environment variables:
  stage: build
  script:
    - echo "Do a test here"
    - echo "Here are some default, global, & local variables..."
    - echo $CICOMMITSHORTSHA
    - echo $grouplevelvariable
```


- echo \$projectlevelvariable
- echo \$INLINEGLOBALVARIABLE
- echo \$INLINELOCALVARIABLE

1. After doing this, your .gitlab-ci.yml file will look like this:

```

yml
stages:
  - test
  - build

test job:
  stage: test
  script:
    - echo "I am a unit test!"

build job:
  stage: build
  script:
    - echo "I am a build image!"

environment echoes:
  stage: build
  script:
    - echo "Who am I running as..."
    - whoami
    - echo "Where am I..."
    - pwd
    - ls -al
    - echo "Here's what is available in our environment..."
    - env

environment variables:
  stage: build
  script:
    - echo "Do a test here"
    - echo "Here are some default, global, & local variables..."
    - echo $CICOMMITSHORTSHA
    - echo $grouplevelvariable
    - echo $projectlevelvariable
    - echo $INLINEGLOBALVARIABLE
    - echo $INLINELOCALVARIABLE

```

1. Near the top of your .gitlab-ci.yml, in a new line below the entire stages section, paste the following to declare a global inline variable:

```

yml
variables:

```

INLINEGLOBALVARIABLE: "I'm an inline variable set at the global level of the CI/CD configuration file"

> A variable declared at the top level is globally available. In this example, all jobs can use the INLINEGLOBALVARIABLE variable.

1. Inside the environment variables job, just below that job's stage: build line (but before the script line), paste the following to declare a local inline variable. The variables keyword should be at the same indentation as that job's stage and script keywords.

```
yml
variables:
  INLINELOCALVARIABLE: "I'm an inline variable set at the job level of the CI/CD
configuration file"
```

> Since this variable inside a job, it is only accessible by the job. For this example, INLINELOCALVARIABLE is only accessible in the environment variables job.

1. At this point, your .gitlab-ci.yml file will look like this:

```
yml
stages:
  - test
  - build

variables:
  INLINEGLOBALVARIABLE: "I'm an inline variable set at the global level of the CI/CD
configuration file"

test job:
  stage: test
  script:
    - echo "I am a unit test!"

build job:
  stage: build
  script:
    - echo "I am a build image!"

environment echoes:
  stage: build
  script:
    - echo "Who am I running as..."
    - whoami
    - echo "Where am I..."
    - pwd
    - ls -al
    - echo "Here's what is available in our environment..."
```

- env

environment variables:

stage: build

variables:

 INLINELOCALVARIABLE: "I'm an inline variable set at the job level of the CI/CD configuration file"

script:

- echo "Do a test here"
- echo "Here are some default, global, & local variables..."
- echo \$CICOMMITSHORTSHA
- echo \$grouplevelvariable
- echo \$projectlevelvariable
- echo \$INLINEGLOBALVARIABLE
- echo \$INLINELOCALVARIABLE

> When defining variables, watch your indentation. Global variables must be indented by 2 spaces, and must be immediately under a flush-left variables keyword that is outside any job definition. Local variables must be indented 4 spaces, and must be immediately under a variables keyword that is indented 2 spaces and is within a job definition.

1. In the Commit message field, type Add custom variables, ensure the Target Branch set to main, and click Commit changes.

Task B: Add Group and Project-level Variables

> There are two types of variables we will discuss in this section: group and project-level variables. These variables are accessible at either the group or project level, allowing for a wider application of CI/CD variables.

1. Navigate to your My Test Group group by clicking it from the breadcrumbs at the top of the page.

1. In the left-hand navigation pane, click Settings > CI/CD

1. In the Variables section, click the Expand button.

1. Click Add Variable. Enter grouplevelvariable in the Key dialog box. Hint: watch your spelling, capitalization, and underscores!

1. Type I'm a variable set at the group level in the Value dialog box.

1. Leave all other options at their defaults and click Add variable.

> This is a group level variable, meaning it is accessible to any projects inside of My Test Group.

1. Navigate to your project by clicking your My Test Group in the breadcrumbs at the top of the page, and then clicking on the CICD Demo project.

1. Repeat steps 2 to 6, entering `projectlevelvariable` in the Key field and I'm a variable set at the project level in the Value field.

> This is a project level variable, meaning it is accessible only inside the current project.

1. Setting variables does not trigger a pipeline run, so click Build > Pipelines in the left navigation pane, click the New Pipeline button, and click the Run Pipeline button.

1. Click on the widget for the environment variables job from your running pipeline and verify the variables and their values are correctly displayed in the job output.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/gitlabci cdhandsonlab6.md

> Estimate time to complete: 25 - 30 minutes

Objectives

Job Policy patterns allow the pipeline to control when and if jobs run using the rules keyword. In this lab, you will learn how to create jobs with rules. You will see the impact of these rules on a pipeline and learn how to use variables with pipeline rules.

> Usage of the only and except keywords, while able to accomplish similar results, are not actively being developed and are not encouraged. For more information, [click here](#).

Task A: Creating Jobs with Rules

1. Open your CICD Demo project from previous labs.

1. Click your `.gitlab-ci.yml` file to view its contents. Click Edit > Edit single file.

1. To clean up our configuration file, remove the environment echoes and environment variables jobs. Also remove the variables keyword and the associated variable. After completing these steps, you will have the following `.gitlab-ci.yml` file:

```
yml
stages:
  - test
```

- build

test job:

stage: test

script:

- echo "I am a unit test!"

build job:

stage: build

script:

- echo "I am a build image!"

1. At the end of the file, paste the following code snippet:

yaml

deploy review:

stage: review

script:

- echo "Do your average deploy here"

rules:

- if: '\$CICOMMITREFNAME == "main"'

when: never

- if: '\$CICOMMITTAG'

when: never

- when: always

environment:

name: review/\$CICOMMITREFNAME

deploy release:

stage: deploy

script:

- echo "Deploy to a production environment"

rules:

- if: '\$CICOMMITTAG =~ /^v./'

when: manual

environment:

name: production

deploy staging:

stage: deploy

script:

- echo "Deploy to a staging environment"

rules:

- if: '\$CICOMMITREFNAME == "main"'

when: always

- when: never

environment:

name: staging

1. At the top of .gitlab-ci.yml, in the stages section, add the review and deploy stages.

1. In full, you will have the following .gitlab-ci.yml file upon completion:

```
yml
stages:
  - test
  - build
  - review
  - deploy

test job:
  stage: test
  script:
    - echo "I am a unit test!"

build job:
  stage: build
  script:
    - echo "I am a build image!"

deploy review:
  stage: review
  script:
    - echo "Do your average deploy here"
  rules:
    - if: '$CICOMMITREFNAME == "main"'
      when: never
    - if: '$CICOMMITTAG'
      when: never
    - when: always
  environment:
    name: review/$CICOMMITREFNAME

deploy release:
  stage: deploy
  script:
    - echo "Deploy to a production environment"
  rules:
    - if: '$CICOMMITTAG =~ /^v./'
      when: manual
  environment:
    name: production

deploy staging:
  stage: deploy
  script:
    - echo "Deploy to a staging environment"
```

```
rules:
  - if: '$CICOMMITREFNAME == "main"'
    when: always
  - when: never
environment:
  name: staging
```

1. In the Commit message field, type Add CI structure job definitions, ensure the Target Branch is set to main, and click Commit changes.

1. In the left-hand navigation pane, click Build > Pipelines and click the status icon next to the most recent pipeline run.

1. Click the widgets to see what environment the pipeline is deploying the code to. In the left sidebar, click Operate > Environments to see the environments that have been created.

> You will see that deploy staging is the only one of the three jobs that executed, based on the rules that were defined for each job.

1. Optional: Experiment with triggering pipelines using different branches and tags. Can you get different pipeline runs to execute the deploy release, deploy review, and deploy staging jobs?

> Hint: Look at the rules keyword in the relevant .gitlab-ci.yml job definitions.

Solutions

Task B1: Running the deploy review Job

1. Review the rules specified in the deploy review's rules section. It will only run when A) The branch name (represented by \$CICOMMITREFNAME) is not equal to main, and B) there is no tag on the branch (represented by \$CICOMMITTAG).

> A variable used with an if keyword on its own is checking if said variable has any value associated with it. If it has any value, regardless of what that value is, the statement is true. This includes values that would be considered false in other programming languages, such as False. If there is no value, the statement is false. A variable with whitespace as its value is considered false as well.

1. Create a new branch by clicking on Code > Branches.

1. Click the New branch button.

1. Type Dev in the branch name section, and click Create branch.

1. Click on Build > Pipelines.

1. Click on the Run Pipeline button.

1. Under Run for branch name or tag, make sure Dev is selected.

1. Click on the Run Pipeline button.

Your deploy review job should be the only job that should be running.

Task B2: Running the deploy release Job

1. Open your .gitlab-ci.yml file and review the rules specified in the deploy release rules section. It will only run when A) The branch name (represented by \$CI_COMMIT_REFNAME) is not equal to main, and B) there is a tag on the branch that begins with the letter v (represented by \$CI_COMMIT_TAG =~ /^v./)

1. We will need to make a tag for this job to run. To make a tag, click on Code > Tags.

1. Click on the New tag button.

1. Type in v1.0 into the Tag name section.

1. Change the Create from option to display Dev instead.

1. Click the Create tag button.

1. Click on Build > Pipelines.

1. Click on the Run Pipeline button.

1. Under Run for branch name or tag, make sure v1.0 is selected.

1. Click on the Run Pipeline button.

1. Your deploy release job should be the only job available. Since the job has been set to manual, it will not run until you explicitly start it. Click on the arrow next to the job to start the job.

Task B3: Running the deploy staging Job

1. Open your .gitlab-ci.yml file and review the rules specified in the deploy staging rules section. It will only run when A) The branch name (represented by \$CI_COMMIT_REFNAME) is equal to main.

1. In the top left corner, click on the button that says Dev, and set the option to be main instead.

1. Click on Build > Pipelines.

1. Click on the Run Pipeline button.

1. Under Run for branch name or tag, make sure main is selected.

1. Click on the Run Pipeline button.

Your deploy staging job should be the only job that is running.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/gitlabci cdhandsonlab7.md

> Estimate time to complete: 20 - 25 minutes

Objectives

Artifacts in GitLab are files that are created in a job, and then passed to other jobs in later stages. An artifact in one stage cannot be passed to a job in the same stage. You can later access and download any artifacts created in a pipeline. For more information, [click here](#).

In this lab, you will learn how to create an artifact with your .gitlab-ci.yml file. After creating the artifact, you will view the artifact in the GitLab UI.

Task A: Add a main.go File

1. Open your CICD Demo project from previous labs.
1. Navigate to Code > Repository and add a new file by clicking + > This directory > New file
1. In the Filename field, enter main.go
1. Paste the following code into the file.

```
go
package main

import (
    "fmt"
    "net/http"
)

func helloworld() string {
    return "Hello World!!"
}
```

```

func healthcheck() string {
    return "Health OK!"
}

func livenesscheck() string {
    return "I am alive!!!"
}

func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r http.Request) {
        fmt.Fprintf(w, helloworld())
    })

    http.HandleFunc("/healthz", func(w http.ResponseWriter, r http.Request) {
        fmt.Fprintf(w, healthcheck())
    })

    http.HandleFunc("/liveness", func(w http.ResponseWriter, r http.Request) {
        fmt.Fprintf(w, livenesscheck())
    })

    http.ListenAndServe(":8080", nil)
}

```

1. In the Commit message field, type Add main.go file, ensure the Target Branch is set to main, and click Commit changes.

Task B: Add Artifacts to your Pipeline

1. In the left sidebar, click Code > Repository.
1. Select your .gitlab-ci.yml file to view its contents. Click Edit > Edit single file. Paste the following snippet at the end of the file.

```

yml
build app:
  image: golang:latest
  stage: build
  script:
    - go build -o app main.go
  artifacts:
    paths:
      - app
    expirein: 1 hour

```

1. In the Commit message field, type Add CI artifacts, ensure the Target Branch is set to main, and click Commit changes.

1. In the left-hand navigation pane, click Build > Pipelines and click the status icon for the most recent pipeline run.

1. When the build app job finishes, click it to review the job's output log.

> If the job fails with a message about being unable to find go.mod, retry the job until it passes. This is an intermittent Go build bug.

1. In the Job artifacts panel on the right of the page, click Browse and notice that the app artifact created by the build app pipeline job is available for download.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/gitlabci cdhandsonlab8.md

> Estimate time to complete: 15 - 20 minutes

Objectives

Docker is a platform commonly used by developers to build container applications. It is possible to generate Docker containers as a part of a CI/CD build process. In this lab, you will learn how to define a Dockerfile for your project, and integrate it into your .gitlab-ci.yml file.

Task A: Add a Dockerfile

1. Open your CICD Demo project from earlier labs.

1. Add a new file to the CICD Demo project by selecting + > This directory > New file.

1. In the Filename field, enter Dockerfile.

1. Paste the following into the body of the new file.

```
Dockerfile
FROM golang:1.11-alpine as builder
WORKDIR /usr/build
ADD main.go .
RUN go build -o app .
```

```
FROM alpine:latest
```

```
WORKDIR /usr/src
```

```
COPY --from=builder /usr/build/app .
```

```
EXPOSE 8080
```

```
CMD ["/usr/src/app"]
```

1. In the Commit message field, type Add Dockerfile, ensure the Target Branch is set to main, and click Commit changes.

Task B: Define a build image Job

1. In the left navigation bar, click Code > Repository.

1. Open the .gitlab-ci.yml file and click Edit > Edit single file. Paste the following new job definition at the end of the file.

```
yml
build image:
  stage: build
  image: docker:27
  services:
    - docker:27-dind
  variables:
    IMAGE: $CI_REGISTRYIMAGE/$CI_COMMITREFSLUG:$CI_COMMITSHA
  script:
    - docker login -u $CI_REGISTRYUSER -p $CI_REGISTRYPASSWORD $CI_REGISTRY
    - docker build -t $IMAGE .
    - docker push $IMAGE
```

> There are alternative methods that can be used for building Docker based images. If your runners do not support Docker in Docker, you can consider using a tool like kaniko, as shown in the configuration below:

```
yml
build image:
  stage: build
  image:
    name: gcr.io/kaniko-project/executor:v1.14.0-debug
    entrypoint: [""]
  script:
    - /kaniko/executor
    - --context "${CI_PROJECTDIR}"
    - --dockerfile "${CI_PROJECTDIR}/Dockerfile"
    - --destination "${CI_REGISTRYIMAGE}:${CI_COMMITTAG}"
```

1. In the Commit message field, type Add "build image" job definition, ensure the Target Branch is set to main, and click Commit changes.

Task C: Ensure the Pipeline is Running

1. Go to Build > Pipelines. Click the most recent pipeline run.

1. Click the widget for the build image job to see its progress. Wait for the job to complete.

1. In the left navigation pane, click Deploy > Container Registry and view the container that was just uploaded by the build image job.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/gitlabci cdhandsonlab9.md

> Estimate time to complete: 15 - 20 minutes

Objectives

SAST, an optional feature on CI/CD pipelines, analyzes your source code for known vulnerabilities. GitLab's Vulnerability Report then shows any old or new vulnerabilities found with each pipeline run. In this lab, you will learn the process of enabling SAST scans in your CI/CD pipelines. You can learn more about SAST scanner by clicking [here](#).

Task A. Creating a Test File

1. Open your CI/CD Demo project from previous labs.

1. Near the top left, to the right of the branch dropdown, click (+) > This directory > New file.

1. Type run.py as the name of the file.

1. Copy and paste the following code into the body:

```
python
import subprocess
```

```
ip = input("Enter your server ip: ")
subprocess.run(["ping", ip])

print("Attempting to connect to the server")
print("Application authentication was successful")
```

1. Type Create Run.py as a test file into the commit message.

1. Set the Target branch to main.

1. Click Commit changes.

> This file has a command injection vulnerability, which can lead to security breaches. We are going to use the SAST Scanner to detect issues in our code.

Task B. Creating and Running a SAST Scan

1. Open your CICD Demo project from previous labs.

1. Click on your .gitlab-ci.yml file to view its contents.

1. Click Edit > Edit single file. Paste the following snippet at the end of the file.

```
yml
include:
  - template: Jobs/SAST.gitlab-ci.yml
```

1. In the Commit message field, type Enable SAST, leave the Target Branch set to main, and click Commit changes.

1. Navigate to the pipeline that was started by this change and click the semgrep-sast job to ensure that it's running.

> It might take a minute or two for the Build stage to complete first.

1. To view the results of the SAST scan, click Secure > Vulnerability Report in the left-hand navigation pane. In the Tool drop-down list, select SAST. Click on any vulnerabilities to learn more about them.

Task C. Fix the Key Issues

1. In the sidebar, navigate to Code > Repository.

1. Click the run.py file.

1. On the run.py file, click the blue Edit dropdown and select Edit single file.

1. The scan showed that lines 1 and 4 contained a vulnerability.

```
python
main.py Line 1
import subprocess
```

```
python
main.py Line 4
subprocess.run(["ping", ip])
```

> The problem is that these lines use user input to run a system command. In this situation, a user could craft an input that causes the application to execute an unintended command. To resolve this, you can either remove the command, or remove the user input from the command.

1. Remove the two lines from the code. This is what the file should look like now.

```
python
print("Attempting to connect to the server")
print("Application authentication was successful")
```

1. In the commit field, type Fix changes as suggested by SAST scan, leave the Target Branch set to main, and click Commit changes

> The SAST scan will need to run again in order to update the changes.

1. Once the pipeline has finished running, click Secure > Vulnerability Report in the left-handed navigation pane. In the Tool drop-down list, select SAST. Review to see if the security issues have been remediated.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab CI/CD, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/gitlabci cdhandsonlab9alt.md

> Estimated time to complete: 15 - 20 minutes

Objectives

Code quality scans allow you to analyze your source code for quality and complexity. The goal

of a code quality scan is to keep your project's code simple, readable, and easier to maintain. In this lab, you will learn how to enable and view a code quality scan in your project. To learn more about code quality scans, [click here](#).

Task A. Add a Python file with code quality problems

1. Using the left navigation pane, click Code > Repository.
1. Create a new file by going to the top of the window and clicking + > This directory > New file.
1. For the File name, type HelloWorld.py.
1. Paste the following Python code into the file's contents:

```
python
def helloworld(a, b, c, d, e, f, g):
    print("Hello world")
    TODO: improve this function
```

1. In the Commit message field, type Add Python code.
1. Set the Target Branch to main.
1. Click Commit changes.

Task B. Configure the .gitlab-ci.yml with Code Quality Scanning

1. In the left navigation pane, click Code > Repository.
1. Click on your existing .gitlab-ci.yml file.
1. Edit the file by clicking the blue Edit dropdown and selecting Edit single file.
1. Delete all of the existing code in the file. Replace the code with the following file content:

```
yml
stages:
  - test

test-job:
  stage: test
  script:
    - echo "Pipeline needs at least one job"

include:
  - template: Code-Quality.gitlab-ci.yml
```


TODO: should we refactor this file?

> This code starts by defining a single stage named test. Next, a single job is defined named test-job, which runs as a part of the test stage. The include section enables code quality scanning by including the code quality template. Finally, the TODO command is added as an example of code that is flagged as a code quality problem.

1. In the Commit message field, type Add CI/CD configuration file that includes code quality scanning.

1. Set the Target Branch to main.

1. Click Commit changes.

Task C. View code quality scan results

1. In the left navigation pane, click Build > Pipelines.

1. The top row represents the pipeline that started running when you committed the .gitlab-ci.yml file in the previous section. Wait until the status icon at the left of that pipeline says passed.

> It can take as long as 5 minutes for the code quality scanner to complete in the training environment, so this is a great time to grab a snack.

1. Once the pipeline's status is passed, click the status icon to see the pipeline details.

1. On the pipeline details screen, click the Code Quality tab above the pipeline graph.

1. Notice that the scanner found 3 code quality issues in 2 different files: 2 in HelloWorld.py and 1 in .gitlab-ci.yml.

Task D. Make a branch

1. In the left navigation pane, click Code > Branches.

1. Click New branch. In the Branch name field, type branch-A.

1. Click Create branch.

1. Click Create merge request in the top right of the window. Leave all settings at their default values.

1. Click Create merge request.

Task E. Fix issues on the branch

1. In the left navigation pane, click Code > R